

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"Белгородский государственный технологический университет им. В. Г.
Шухова"
(БГТУ им. В.Г. Шухова)

Институт энергетики, информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники
и автоматизированных систем

Лабораторная работа №2
по дисциплине: Операционные системы
по теме: Процессы и потоки в ОС Linux (Ubuntu): сравнение,
механизмы синхронизации. Парадигмы межпроцессорного
взаимодействия.

Выполнил: студент группы ПВ-221
Дорошенко Владимир Юрьевич
Проверил: доц. Островский Алексей Мичеславович,
асс. Четвертухин Виктор Романович

Цель работы: Изучить различия между процессами и потоками в ОС Linux (Ubuntu), а также освоить механизмы синхронизации и межпроцессорного взаимодействия для обеспечения корректной работы программ в многозадачной среде. Цель работы обуславливает постановку и решение следующих задач:

- 1) Изучить основные концепции, относящиеся к процессам и потокам в ОС Linux (Ubuntu).
- 2) Получить основное понятие о создании потоков в Linux с использованием библиотеки POSIX Threads (pthread.h).
- 3) Изучить основные механизмы синхронизации: мьютексы, семафоры, спинлоки, барьеры, блокировки читателей/писателей, futex, каналы.
- 4) Изучить механизмы межпроцессорного взаимодействия в ОС Linux (Ubuntu).
- 5) Выполнить индивидуальное задание, связанное с применением механизмов синхронизации и межпроцессорного взаимодействия в многозадачной среде.

Информация

Змей Горыныч имеет три головы, каждая из которых независимо от других голов ест продукцию с двух кондитерских фабрик. Каждая фабрика производит разные типы кондитерских изделий (торты, пирожные, конфеты, пряники) с различной скоростью. Головам нужно получать продукцию из общего склада, но склад ограничен по объему. Задача — организовать взаимодействие между фабриками (производителями) и головами Змея Горыныча (потребителями) так, чтобы они корректно синхронизировались при производстве и потреблении продукции, избегая конфликтов, минимизируя ситуации простоя и переполнения склада.

Каждая фабрика производит разные типы продукции с различной скоростью:

Фабрика 1 (производит торты и пирожные):

Выпуск тортов: 1 торт каждые 70 мс.

Выпуск пирожных: 2 пирожных каждые 50 мс.

Фабрика 2 (производит конфеты и пряники):

Выпуск конфет: 3 конфеты каждые 40 мс.

Выпуск пряников: 1 пряник каждые 60 мс.

Вместимость склада: 10 единиц продукции.

Потребление продукции:

Каждая из трех голов Змея Горыныча потребляет продукцию с разной скоростью:

Голова 1:

Потребляет торты и конфеты.

Потребляет 1 продукт каждые 80 мс.

Голова 2:

Потребляет пирожные и пряники.

Потребляет 1 продукт каждые 90 мс.

Голова 3:

Потребляет любые виды продукции (универсальная голова).

Потребляет 1 продукт каждые 70 мс.

Вариант 8

8. Вводится дополнительное условие в логику задачи. Каждая голова Змея Горыныча должна уметь проверять содержимое склада несколько раз подряд, прежде чем выбрать продукт. Для корректной работы программы, проверка и забор продукции должны быть защищены, в случае потоков, рекурсивными мьютексами (`pthread_mutexattr_settype`), а в случае процессов — семафорами (`sys/sem.h`). Голова может вызвать функцию проверки доступности продукта несколько раз (например, чтобы проверить разные типы продукции). Важно, чтобы каждая голова корректно освобождала мьютекс после завершения работы.

Потоки

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define __USE_UNIX98

#define STORAGE_CAPACITY 10

typedef struct {
    char type; // t - торт, p - пирожное, c - конфета, g - пряник
} Product;

Product storage[STORAGE_CAPACITY];
int storage_size = 0;
int storage_start = 0;
int storage_end = 0;

// Мьютекс для синхронизации
pthread_mutex_t storage_mutex;
pthread_cond_t not_empty;
pthread_cond_t not_full;

// Функция добавления продукта на склад
void add_product(Product product) {
    pthread_mutex_lock(&storage_mutex);

    while (storage_size == STORAGE_CAPACITY) {
        pthread_cond_wait(&not_full, &storage_mutex);
    }

    storage[storage_end] = product;
    storage_end = (storage_end + 1) % STORAGE_CAPACITY;
    storage_size++;

    pthread_cond_signal(&not_empty);
    pthread_mutex_unlock(&storage_mutex);
}
```

```

// Функция забора продукта со склада
Product take_product() {
    pthread_mutex_lock(&storage_mutex);

    while (storage_size == 0) {
        pthread_cond_wait(&not_empty, &storage_mutex);
    }

    Product product = storage[storage_start];
    storage_start = (storage_start + 1) % STORAGE_CAPACITY;
    storage_size--;

    pthread_cond_signal(&not_full);
    pthread_mutex_unlock(&storage_mutex);

    return product;
}

// Функция производства продукции для фабрик
void* factory_produce(void* arg) {
    char factory_type = *(char*)arg;

    while (1) {
        Product product;
        if (factory_type == '1') {
            product.type = 't';
            usleep(70000);
            add_product(product);

            product.type = 'p';
            usleep(50000);
            add_product(product);
        } else {
            product.type = 'c';
            usleep(40000);
            add_product(product);

            product.type = 'g';
            usleep(60000);
            add_product(product);
        }
    }
}

// Функция поедания продуктов для голов Змея Горыныча
void* dragon_head_consume(void* arg) {
    char head_type = *(char*)arg;

    while (1) {

```

```

        Product product = take_product();

        // Голова 1: ест торты и конфеты
        if (head_type == '1' && (product.type == 't' || product.type == 'c')) {
            usleep(80000);
            if (product.type == 't') {
                printf("Голова 1 съела продукт: торт\n");
            } else {
                printf("Голова 1 съела продукт: конфету\n");
            }
        }
        else if (head_type == '2' && (product.type == 'p' || product.type ==
'g')) {
            usleep(90000);
            if (product.type == 'p') {
                printf("Голова 2 съела продукт: пирожное\n");
            } else {
                printf("Голова 2 съела продукт: пряник\n");
            }
        }
        else if (head_type == '3') {
            usleep(70000);
            if (product.type == 't') {
                printf("Голова 3 съела продукт: торт\n");
            } else if (product.type == 'c'){
                printf("Голова 3 съела продукт: конфету\n");
            } else if (product.type == 'p') {
                printf("Голова 3 съела продукт: пирожное\n");
            } else {
                printf("Голова 3 съела продукт: пряник\n");
            }
        } else {
            add_product(product);
        }
    }
}

int main() {
    pthread_t factories[2];
    pthread_t heads[3];

    pthread_mutexattr_t mutex_attr;
    pthread_mutexattr_init(&mutex_attr);
    pthread_mutexattr_settype(&mutex_attr, PTHREAD_MUTEX_RECURSIVE);
    pthread_mutex_init(&storage_mutex, &mutex_attr);
    pthread_cond_init(&not_empty, NULL);
    pthread_cond_init(&not_full, NULL);

    char factory1 = '1';
    char factory2 = '2';
    pthread_create(&factories[0], NULL, factory_produce, &factory1);

```

```

pthread_create(&factories[1], NULL, factory_produce, &factory2);

char head1 = '1';
char head2 = '2';
char head3 = '3';
pthread_create(&heads[0], NULL, dragon_head_consume, &head1);
pthread_create(&heads[1], NULL, dragon_head_consume, &head2);
pthread_create(&heads[2], NULL, dragon_head_consume, &head3);

pthread_join(factories[0], NULL);
pthread_join(factories[1], NULL);
pthread_join(heads[0], NULL);
pthread_join(heads[1], NULL);
pthread_join(heads[2], NULL);

return 0;
}

```

Пример работы программы:

```

● wwwxtus@DESKTOP-I1F97A5:~/Универ/Операционные системы/lab2$ gcc lab2.c -o lab2
ⓧ wwwxtus@DESKTOP-I1F97A5:~/Универ/Операционные системы/lab2$ ./lab2
Голова 1 съела продукт: конфету
Голова 3 съела продукт: торт
Голова 2 съела продукт: пряник
Голова 3 съела продукт: пирожное
Голова 1 съела продукт: конфету
Голова 3 съела продукт: торт
Голова 2 съела продукт: пряник
Голова 1 съела продукт: конфету
Голова 3 съела продукт: пирожное
Голова 2 съела продукт: пряник
Голова 1 съела продукт: торт
Голова 3 съела продукт: конфету
Голова 2 съела продукт: пирожное
Голова 3 съела продукт: пряник
Голова 1 съела продукт: торт
Голова 3 съела продукт: конфету
Голова 2 съела продукт: пирожное
Голова 1 съела продукт: конфету
Голова 3 съела продукт: пряник
Голова 2 съела продукт: пирожное
Голова 3 съела продукт: пряник
Голова 1 съела продукт: торт
Голова 3 съела продукт: конфету
Голова 1 съела продукт: торт
Голова 2 съела продукт: пряник
Голова 3 съела продукт: пирожное
Голова 1 съела продукт: конфету
Голова 2 съела продукт: пряник
Голова 3 съела продукт: торт
Голова 1 съела продукт: конфету

```

Процессы

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <semaphore.h>
#include <fcntl.h>
#include <string.h>

#define STORAGE_CAPACITY 10

typedef struct {
    char type; // t - торт, p - пирожное, c - конфета, g - пряник
} Product;

typedef struct {
    Product storage[STORAGE_CAPACITY];
    int storage_size;
    int storage_start;
    int storage_end;
    sem_t not_empty;
    sem_t not_full;
    sem_t mutex;
} SharedStorage;

SharedStorage *initialize_shared_storage() {
    SharedStorage *shared = mmap(NULL, sizeof(SharedStorage),
                                  PROT_READ | PROT_WRITE,
                                  MAP_SHARED | MAP_ANONYMOUS, -1, 0);

    if (shared == MAP_FAILED) {
        perror("mmap");
        exit(EXIT_FAILURE);
    }

    shared->storage_size = 0;
    shared->storage_start = 0;
    shared->storage_end = 0;

    sem_init(&shared->mutex, 1, 1); // мьютекс для доступа к складу
    sem_init(&shared->not_empty, 1, 0); // счетчик, когда склад не пуст
    sem_init(&shared->not_full, 1, STORAGE_CAPACITY); // счетчик, когда склад не
    полный

    return shared;
}

void destroy_shared_storage(SharedStorage *shared) {
    sem_destroy(&shared->mutex);
    sem_destroy(&shared->not_empty);
}
```



```

    sem_destroy(&shared->not_full);
    munmap(shared, sizeof(SharedStorage));
}

// Добавление продукта на склад
void add_product(SharedStorage *shared, Product product) {
    sem_wait(&shared->not_full);
    sem_wait(&shared->mutex);

    shared->storage[shared->storage_end] = product;
    shared->storage_end = (shared->storage_end + 1) % STORAGE_CAPACITY;
    shared->storage_size++;

    sem_post(&shared->mutex);
    sem_post(&shared->not_empty);
}

// Забор продукта со склада
Product take_product(SharedStorage *shared) {
    sem_wait(&shared->not_empty);
    sem_wait(&shared->mutex);

    Product product = shared->storage[shared->storage_start];
    shared->storage_start = (shared->storage_start + 1) % STORAGE_CAPACITY;
    shared->storage_size--;

    sem_post(&shared->mutex);
    sem_post(&shared->not_full);

    return product;
}

// Производство продукции фабрикой
void factory_produce(SharedStorage *shared, char factory_type) {
    while (1) {
        Product product;
        if (factory_type == '1') {
            product.type = 't';
            usleep(70000);
            add_product(shared, product);

            product.type = 'p';
            usleep(50000);
            add_product(shared, product);
        } else {
            product.type = 'c';
            usleep(40000);
            add_product(shared, product);

            product.type = 'g';
            usleep(60000);

```

```

        add_product(shared, product);
    }
}

// Потребление продукции головами Змея Горыныча
void dragon_head_consume(SharedStorage *shared, char head_type) {
    while (1) {
        Product product = take_product(shared);

        if (head_type == '1' && (product.type == 't' || product.type == 'c')) {
            usleep(80000);
            if (product.type == 't') {
                printf("Голова 1 съела продукт: торт\n");
            } else {
                printf("Голова 1 съела продукт: конфету\n");
            }
        } else if (head_type == '2' && (product.type == 'p' || product.type ==
'g')) {
            usleep(90000);
            if (product.type == 'p') {
                printf("Голова 2 съела продукт: пирожное\n");
            } else {
                printf("Голова 2 съела продукт: пряник\n");
            }
        } else if (head_type == '3') {
            usleep(70000);
            if (product.type == 't') {
                printf("Голова 3 съела продукт: торт\n");
            } else if (product.type == 'c') {
                printf("Голова 3 съела продукт: конфету\n");
            } else if (product.type == 'p') {
                printf("Голова 3 съела продукт: пирожное\n");
            } else {
                printf("Голова 3 съела продукт: пряник\n");
            }
        } else {
            add_product(shared, product);
        }
    }
}

int main() {
    SharedStorage *shared = initialize_shared_storage();

    pid_t factories[2], heads[3];

    factories[0] = fork();
    if (factories[0] == 0) {
        factory_produce(shared, '1');
        exit(0);
    }
}

```

```

    }

    factories[1] = fork();
    if (factories[1] == 0) {
        factory_produce(shared, '2');
        exit(0);
    }

    heads[0] = fork();
    if (heads[0] == 0) {
        dragon_head_consume(shared, '1');
        exit(0);
    }

    heads[1] = fork();
    if (heads[1] == 0) {
        dragon_head_consume(shared, '2');
        exit(0);
    }

    heads[2] = fork();
    if (heads[2] == 0) {
        dragon_head_consume(shared, '3');
        exit(0);
    }

    for (int i = 0; i < 2; i++) {
        waitpid(factories[i], NULL, 0);
    }

    for (int i = 0; i < 3; i++) {
        waitpid(heads[i], NULL, 0);
    }

    destroy_shared_storage(shared);

    return 0;
}

```

Пример работы программы:

```
wwwxtus@DESKTOP-I1F97A5:~/Универ/Операционные системы/lab2$ ./lab2_processes
Голова 1 съела продукт: конфету
Голова 3 съела продукт: торт
Голова 2 съела продукт: пряник
Голова 3 съела продукт: пирожное
Голова 1 съела продукт: конфету
Голова 3 съела продукт: торт
Голова 2 съела продукт: пряник
Голова 1 съела продукт: конфету
Голова 3 съела продукт: пирожное
Голова 2 съела продукт: пряник
Голова 1 съела продукт: торт
Голова 3 съела продукт: конфету
Голова 2 съела продукт: пирожное
Голова 3 съела продукт: пряник
Голова 1 съела продукт: торт
Голова 3 съела продукт: конфету
Голова 2 съела продукт: пирожное
Голова 1 съела продукт: конфету
Голова 3 съела продукт: пряник
Голова 2 съела продукт: пирожное
Голова 1 съела продукт: торт
Голова 3 съела продукт: пряник
Голова 3 съела продукт: конфету
Голова 1 съела продукт: торт
Голова 2 съела продукт: пряник
Голова 3 съела продукт: пирожное
Голова 1 съела продукт: конфету
Голова 2 съела продукт: пряник
Голова 3 съела продукт: торт
```

Вывод

В ходе работы были изучены основные различия между процессами и потоками в операционной системе Linux (Ubuntu), а также практические аспекты их применения в многозадачной среде. Процессы, как независимые экземпляры выполнения программ, обладают собственными адресными пространствами и системными ресурсами, что обеспечивает изоляцию, но увеличивает затраты на создание и переключение контекста. Потоки, в свою очередь, представляют собой более легковесные сущности, которые разделяют ресурсы одного процесса, позволяя эффективно управлять многозадачностью и уменьшать издержки на создание и переключение контекста. Работа позволила освоить практические аспекты многозадачности и межпроцессного взаимодействия, включая создание потоков и процессов, а также применение различных механизмов синхронизации. Эти навыки обеспечат возможность разрабатывать корректные и эффективные программы, способные работать в многозадачной среде.